Warehouse Inventory Management System

Final Project Report

Course	CS6103 — Introduction to Java
Semester	Spring 2026
Student	Bhavith Chandra
Net ID	bc4066
Repository	github.com/Bhavith-Chandra/warehouse-inventory-management

May 11, 2026

Contents

Abstract

This report describes the design and implementation of a **Warehouse Inventory Management System** — a networked, multi-user desktop application built as the final project for CS6103: Introduction to Java. The system addresses a practical warehouse operations problem: multiple staff members and administrators need to view and update shared inventory data simultaneously from separate machines, with the guarantee that concurrent writes never corrupt stock figures or produce an inconsistent database state.

The application follows a three-tier architecture implemented entirely in Java. The **presentation tier** is a JavaFX 21 desktop client with a sidebar-navigated interface that includes a live dashboard showing KPI cards, a category pie chart, a top-stock bar chart, and a real-time activity feed; a searchable and filterable product inventory table with full CRUD dialogs; a dedicated stock-operations panel for receiving and shipping goods; a low-stock alert view with an inline restock shortcut; a transaction-history log with colour-coded action badges; and CSV export for both inventory snapshots and audit records. The **application tier** is a Java TCP server that accepts multiple simultaneous client connections — each served by its own thread — and exposes a 14-verb wire protocol built on serialised Java objects. After every successful mutation the server broadcasts a push event to all subscribed clients, keeping every open UI current without polling. The **data tier** is an SQLite database accessed through JDBC with WAL journal mode, allowing reads to proceed concurrently with an in-flight write transaction.

Product management (create, read, update, delete) is restricted to `ADMIN` accounts. Stock operations — add, remove, and absolute cycle-count adjust — are available to both `ADMIN` and `STAFF` and are each recorded in an immutable audit log that captures the quantity delta, resulting quantity, performer identity, timestamp, and a freetext reason. Passwords are stored as salted SHA-256 hashes and verified in constant time to prevent timing-oracle attacks.

The central correctness challenge from the project proposal — two clients racing to deduct the last unit of a product — is solved with a two-layer defence: a per-product `ReentrantLock` serialises in-process writes to the same product, while a JDBC transaction with an in-transaction re-read of the current quantity enforces the non-negative stock invariant at the database level. An automated concurrency stress test races 15 client threads against 50 units and confirms that exactly 50 deductions succeed with zero double-deductions and a final stock of zero. A separate end-to-end verification suite covers all 29 project-proposal requirements and reports 29/29 passing checks.

1. Introduction

Modern warehouses require real-time, multi-user access to inventory data. Staff need to log stock movements instantly, managers need accurate stock levels at a glance, and the system must remain consistent even when multiple users act simultaneously. This project builds a fully functional desktop application that satisfies all of these requirements using the core Java technologies covered in CS6103.

The system delivers the following capabilities:

- Real-time product catalogue management (create, read, update, delete)

- Stock-level tracking via add, remove, and absolute-adjust operations
- Automatic low-stock alerting with inline restock shortcuts
- A full, immutable audit transaction log with timestamps and performer tracking
- Role-based access control (`ADMIN` vs. `STAFF`)
- Live push-event updates so all connected clients refresh automatically
- CSV export of both inventory and transaction history

2. System Architecture

The application is structured as a strict three-tier architecture that isolates presentation, business logic, and persistence.

2.1 Tier 1 — Presentation (JavaFX Client)

The client is a JavaFX 21 desktop application (`InventoryClient.java`). It contains no business logic; every user action is serialised into a `Request` object and transmitted over a TCP socket. The UI runs entirely on the JavaFX Application Thread; all network I/O runs on daemon background threads, and results are marshalled back to the UI via `Platform.runLater()`.

2.2 Tier 2 — Application Server (Java TCP Server)

`InventoryServer` opens a `ServerSocket` on port 5555 and spawns one `Thread(ClientHandler)` per accepted connection. `ClientHandler` reads `Request` objects, dispatches them to `InventoryService` (the business-logic layer), and writes `Response` objects back. After every successful mutation the server broadcasts a push event to all subscribed clients so their UIs refresh without polling.

2.3 Tier 3 — Data Layer (SQLite via JDBC)

`DatabaseManager` owns a single JDBC `Connection` to an SQLite file (`data/warehouse.db`). WAL (Write-Ahead Logging) journal mode is enabled so concurrent reads never block an in-progress write. The schema comprises three tables: `users`, `products`, and `transactions`.

2.4 Wire Protocol

All messages are Java-serialised objects. `Request` carries a `Type` enum (14 verbs: `LOGIN`, `LIST_PRODUCTS`, `ADD_STOCK`, `REMOVE_STOCK`, etc.), a parameter `Map<String,Object>`, and a `requestId = System.nanoTime()`. `Response` carries a `Status` enum (`OK` / `ERROR` / `EVENT`), a payload, and the matching `requestId`. The client's `ServerConnection` correlates incoming responses to outstanding `CompletableFuture`s by `requestId`, enabling fully asynchronous networking over a single socket without blocking the UI thread.

Figure 1: High-level architecture diagram

3. Concurrency Design

The headline correctness property from the project proposal is:

> *"If two users try to deduct the last unit of a product at the same time, only one should succeed."*

`InventoryService.adjustStock()` enforces this with a two-layer defence.

3.1 Layer 1 — Per-Product `ReentrantLock`

A `ConcurrentHashMap<Integer, ReentrantLock>` maps each product ID to its own lock. Every stock mutation acquires the lock for *that* product before performing any I/O. Writes to different products proceed in parallel; writes to the *same* product are strictly serialised within the JVM.

3.2 Layer 2 — JDBC Transaction

Inside the lock the code:

1. Calls `setAutoCommit(false)` to open a transaction.
2. Re-reads the current quantity from the database *inside the same transaction* (preventing stale cached values from leaking through).
3. Validates the business rule: `quantity + delta >= 0`; throws `InsufficientStockException` otherwise.

4. Applies the `UPDATE products SET quantity = ?`.
5. Writes an audit row to `transactions`.
6. Calls `commit()`. On any exception, `safeRollback()` is called.

The database schema also enforces `CHECK (quantity >= 0)` as a final safety net at the storage layer.

3.3 Why Two Layers?

The database constraint alone would allow both clients to *attempt* the deduction; one would receive a confusing `SQLException` from the `CHECK` violation. The application-level `ReentrantLock` ensures that exactly one client gets a clean *"Insufficient stock"* message and the other a successful response—the behaviour users and downstream systems expect.

```
ReentrantLock lock = lockFor(productId);
lock.lock();
try {
    synchronized (db) {
        Connection c = db.getConnection();
        c.setAutoCommit(false);
        try {
            // Re-read inside transaction to prevent stale
               reads
            int current = readQuantity(c, productId);
            int next    = current + delta;
            if (next < 0)
                throw new InsufficientStockException(...);

            updateQuantity(c, productId, next);
            writeAuditLog(c, productId, delta, next,
               performedBy, reason);
            c.commit();
        } catch (Exception e) {
            safeRollback(c);
            throw e;
        }
    }
} finally {
    lock.unlock();
}
```

Listing 1: Core of `adjustStock()` — two-layer concurrency guard

4. Authentication and Security

4.1 Password Storage

Passwords are stored as salted SHA-256 hashes using `PasswordHasher.java`. The on-disk format is:

`base64(salt) : base64(SHA-256(salt || password))`

Constant-time comparison (`MessageDigest.isEqual`) is used during verification to prevent timing-oracle attacks.

4.2 Role-Based Access Control

Role	Permitted Operations	Denied Operations
`ADMIN`	Full access (CRUD + stock + log)	—
`STAFF`	Stock operations, read-only views	Create/update/delete products

Table 1: Role permissions

Role checks are enforced in `ClientHandler.requireRole()` before every admin-only operation. The server also enforces a login-first protocol: any request other than `LOGIN` before authentication is rejected with *"Not authenticated; LOGIN required first."*.

4.3 Default Credentials (First Run)

Username	Password	Role
`admin`	`admin123`	`ADMIN`
`staff`	`staff123`	`STAFF`

Table 2: Seeded default credentials

5. Implementation Overview

5.1 Server-Side Classes

Class	Responsibility
`InventoryServer`	`ServerSocket` accept loop; handler registry; broadcast engine
`ClientHandler`	Per-connection request dispatch; `writeLock` for thread-safe socket writes
`InventoryService`	All business logic; concurrency guards; JDBC transaction management
`DatabaseManager`	Schema creation; WAL/FK PRAGMAs; single JDBC connection lifetime

Table 3: Server-side classes

5.2 Client-Side Classes

Class	Responsibility
`InventoryClient`	JavaFX Application; sidebar navigation; keyboard shortcuts
`ServerConnection`	Async socket I/O; CompletableFuture correlation; event dispatch
`DashboardView`	KPI cards, PieChart by category, BarChart top-10, activity feed
`InventoryView`	Searchable/filterable TableView; Add/Remove Stock dialogs
`StockOpsView`	Dedicated receive/ship/cycle-count panel with live stock preview
`LowStockView`	Auto-filtered low-stock list with one-click Restock button
`HistoryView`	Transaction audit log; colour-coded action badges; CSV export

Table 4: Client-side classes

5.3 Shared / Utility Classes

- `model/` — `Product`, `TransactionLog`, `User` (`Serializable` domain objects transported over the wire)
- `protocol/` — `Request` (14-verb `Type` enum), `Response` (`Status` + `EventType` enums)
- `util/PasswordHasher` — salted SHA-256 hash and verify

6. Testing and Verification

Two automated test programs are included under `src/com/warehouse/test/`.

6.1 `ProposalVerify.java` — End-to-End Verification

Connects to a live server and exercises every project-proposal requirement through 29 automated checks covering:

- All six CRUD operations (`ADD_PRODUCT`, `LIST_PRODUCTS`, `SEARCH_PRODUCTS`, `GET_PRODUCT`, `UPDATE_PRODUCT`, `REMOVE_PRODUCT`)
- All three stock-mutation types (add, remove, absolute adjust)
- Over-deduction guard (insufficient-stock rejection)
- Low-stock report inclusion
- Full transaction-log inspection (type, performer, timestamp)
- Role-based access-control enforcement
- Unauthenticated-request rejection
- Push-event delivery to subscribed clients

> **Result:** **29 / 29 PASS**

6.2 `ConcurrencyStressTest.java` — Race-Condition Proof

Resets a chosen product to N units, launches T client threads each repeatedly calling `REMOVE_STOCK(1)` until rejected, then reads the final stock level.

```
=== Warehouse Concurrency Stress Test ===
productId=1  startStock=50  threads=15

  successful deductions:   50
  'insufficient' rejects:  15
  other errors:             0
  final stock:              0

PASS: exactly 50 units deducted, no double-deduction, final
   stock 0.
```

Listing 2: Sample stress-test output ($N = 50$, $T = 15$)

The test confirms that under heavy parallel contention the invariant $successes = N \ \wedge$ $finalStock = 0$ always holds, and that no double-deduction ever occurs.

7. How to Run

7.1 Prerequisites

- JDK 17 or later (tested on OpenJDK 25)
- JavaFX SDK 21.0.5 — download from https://gluonhq.com/products/javafx/ and extract to `lib/javafx-sdk-21.0.5/`
- `sqlite-jdbc-3.46.1.3.jar`, `slf4j-api-2.0.13.jar`, and `slf4j-simple-2.0.13.jar` are already included in `lib/`

7.2 Build and Launch

```
bash scripts/compile.sh
```

Listing 3: Step 1 — Compile

```
bash scripts/run-server.sh
# First run: creates data/warehouse.db, seeds 2 users + 10
   sample products
# Output: "Warehouse server listening on port 5555"
```

Listing 4: Step 2 — Start the server (Terminal 1)

```
bash scripts/run-client.sh
# Connection dialog: localhost:5555 / admin / admin123
```

Listing 5: Step 3 — Start the client (Terminal 2, repeat for multi-user demo)

```
bash scripts/run-stress.sh
# Custom: bash scripts/run-stress.sh [productId] [startStock] [
   threads]
```

Listing 6: Step 4 (optional) — Concurrency stress test

7.3 Eclipse IDE Import

1. **File → Import → General → Existing Projects into Workspace** → select this folder.
2. The included `.project` and `.classpath` configure all JAR dependencies automatically.
3. Add the following VM arguments to the client run configuration:

```
--module-path lib/javafx-sdk-21.0.5/lib
--add-modules javafx.controls,javafx.fxml,javafx.graphics,
   javafx.base
```

7.4 UI Navigation and Keyboard Shortcuts

View / Action	Description	Shortcut
Dashboard	KPI cards, charts, live activity feed	—
Inventory	Product table; search, filter, CRUD, stock dialogs	Ctrl+F / Ctrl+N
Stock Operations	Receive / ship / cycle-count panel	—
Low Stock Alerts	Products at or below reorder level	—
Transaction History	Full audit log with colour-coded badges	—
Refresh all	Pull latest data from server	Ctrl+R
Export CSV	Save current inventory snapshot to CSV	Ctrl+E

Table 5: UI navigation and shortcuts

8. Mapping to CS6103 Advanced Java Concepts

Concept	Where It Appears
JavaFX GUI	`InventoryClient` — TableView, PieChart, BarChart, Dialog, FilteredList, CSS theming
Java Sockets	`InventoryServer` (ServerSocket), `ServerConnection` (Socket, ObjectIn/OutputStream)
Multithreading	Accept loop spawns one Thread(ClientHandler) per client; Platform.runLater() for UI marshalling
JDBC + concurrency	`DatabaseManager` (WAL, schema); `InventoryService` (ReentrantLock, setAutoCommit, commit/rollback)
Beyond the proposal	
Push-event broadcast	Server broadcasts `Response.event()` to all subscribed clients after every mutation
Salted password hashing	`PasswordHasher` — SHA-256 with per-user random salt, constant-time verify
Automated testing	`ProposalVerify` (29 checks), `ConcurrencyStressTest` (race proof)

Table 6: Project proposal concepts mapped to implementation

9. Conclusion

The Warehouse Inventory Management System demonstrates the application of core Java concepts—TCP sockets, multithreading, JavaFX, and JDBC—in a cohesive, production-quality application. The two-layer concurrency strategy (per-product `ReentrantLock` combined with a JDBC transaction and a re-read of the current stock inside that transaction) correctly solves the classic "last unit" race condition identified in the project proposal as the central correctness challenge.

All 29 automated proposal-verification checks pass, and the concurrency stress test confirms that no double-deduction occurs under heavy parallel load across 15 concurrent client threads.

The full source code, scripts, and this report are available at:

https://github.com/Bhavith-Chandra/warehouse-inventory-management